DISEÑO DE ALGORITMOS

ASESOR:
ING. GERARDO ACEVEDO VEGA

UNIDAD 3 · ACTIVIDAD 3

ALGORITMO EN PSEUDOCÓDIGO: NÚMERO PAR O IMPAR

13 DE OCTUBRE DE 2026

GRUPO 8191 · INFORMÁTICA

Alumno: Mauro Noel Sánchez Campos

Universidad Nacional Autónoma de México

Facultad de Contaduría y Administración

Introducción

Un número entero es par cuando se puede dividir entre 2 sin que sobre nada, y es impar cuando al dividirlo entre 2 sobra 1. Esa idea tan simple es justo lo que necesita la computadora para resolver el problema: no "ve" si un número es par, pero sí puede calcular el residuo de una división. En esta actividad diseño un algoritmo en pseudocódigo que pide un número al usuario y le dice si es par o impar, y después describo el proceso que seguí para construirlo, usando la sintaxis de pseudocódigo del apunte (Kanagusico et al., 2026).

1. Algoritmo en pseudocódigo

```
ALGORITMO ParImpar
VAR
    ENTERO numero, residuo;
INICIO
    ESCRIBIR("Introduce un número entero: ");
    LEER(numero);
    residuo <- numero MOD 2;
    SI residuo = 0 ENTONCES
        ESCRIBIR("El número ", numero, " es PAR");
    SI NO
        ESCRIBIR("El número ", numero, " es IMPAR");
    FIN SI
FIN
```

`MOD` es el operador que devuelve el residuo de una división entera. Por ejemplo, 7 MOD 2 = 1 porque 7 entre 2 es 3 y sobra 1, y 10 MOD 2 = 0 porque la división es exacta.

2. Descripción del proceso

Paso 1. Análisis del problema

Antes de escribir el algoritmo identifiqué tres cosas, como se hace en la fase de análisis del ciclo de vida de un programa (Kanagusico et al., 2026):

Elemento	En este problema
Entrada	Un número entero que escribe el usuario con el teclado.
Proceso	Calcular el residuo de dividir el número entre 2 y compararlo con 0.
Salida	Un mensaje en pantalla que indica si el número es par o impar.

Paso 2. Declaración de variables

Uso dos variables de tipo **ENTERO**: `numero`, para guardar lo que captura el usuario, y `residuo`, para guardar el resultado de la operación. Las declaro como enteras porque la paridad solo tiene sentido en números enteros; un número como 4.5 no es par ni impar.

Paso 3. Entrada de datos

Con `ESCRIBIR` muestro un mensaje que le indica al usuario qué debe hacer, y con `LEER` capturo el valor y lo guardo en `numero`. Si no mostrara el mensaje, el usuario vería la pantalla esperando sin saber qué escribir.

Paso 4. Proceso

La instrucción `residuo <- numero MOD 2` calcula el residuo y lo asigna a la variable con el operador de asignación `<-`. Ese residuo solo puede valer 0 o 1 (o −1 si el número es negativo, como explico más abajo).

Paso 5. Decisión

Uso una estructura selectiva `SI ... ENTONCES ... SI NO ... FIN SI`, porque el algoritmo tiene que tomar uno de dos caminos. Si el residuo es igual a 0, el número es par; en cualquier otro caso, es impar. Preferí preguntar por `residuo = 0` y no por `residuo = 1`: en lenguajes como C el residuo de un número negativo impar es −1 (por ejemplo, −3 % 2 = −1), y si preguntara por 1 el algoritmo diría que −3 es par, lo cual es un error. Preguntando por 0 funciona para positivos, negativos y el cero.

Paso 6. Salida y fin

Dentro de cada camino hay un `ESCRIBIR` que muestra el resultado junto con el número que se capturó, para que el usuario pueda confirmar que se evaluó el valor correcto. La etiqueta `FIN` cierra el algoritmo.

3. Diagrama de flujo

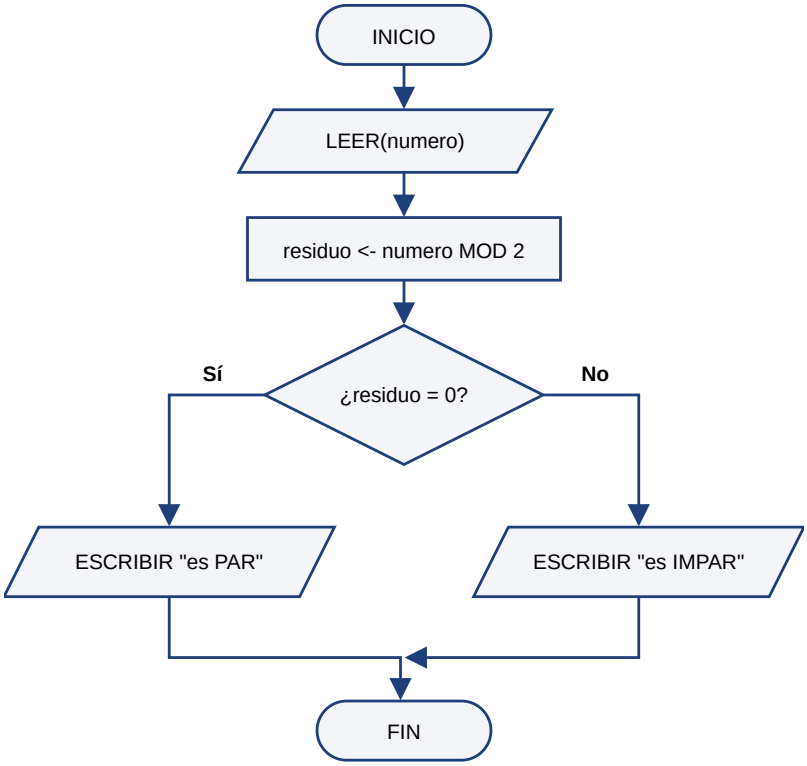


Figura 1. Diagrama de flujo del algoritmo ParImpar. Elaboración propia.

4. Prueba de escritorio

Para comprobar que el algoritmo funciona, lo seguí a mano con varios valores, incluyendo casos especiales como el cero y un número negativo:

numero	residuo = numero MOD 2	¿residuo = 0?	Salida
7	1	No	El número 7 es IMPAR
10	0	Sí	El número 10 es PAR
0	0	Sí	El número 0 es PAR
−3	−1 (en C)	No	El número −3 es IMPAR
128	0	Sí	El número 128 es PAR

En todos los casos el resultado fue correcto. El cero sale como par, lo cual es correcto matemáticamente porque 0 entre 2 es 0 y no sobra nada.

5. Código equivalente en C

Como en la unidad se trabaja con lenguaje C, así quedaría el mismo algoritmo traducido. En C el operador de residuo es `%` y la comparación de igualdad es `==`:

```
#include <stdio.h>

int main(void) {
    int numero, residuo;
    printf("Introduce un numero entero: ");
    scanf("%d", &numero);
    residuo = numero % 2;
    if (residuo == 0)
        printf("El numero %d es PAR\n", numero);
    else
        printf("El numero %d es IMPAR\n", numero);
    return 0;
}
```

Conclusión

Aunque el problema es sencillo, me sirvió para practicar el orden que se sigue al diseñar un algoritmo: primero entender qué entra y qué sale, luego declarar las variables, y después elegir la estructura que resuelve la decisión. Lo que más me dejó fue el detalle de los números negativos: un algoritmo que parece correcto puede fallar con un caso que no se probó, y por eso la prueba de escritorio con valores especiales es parte del proceso y no un paso opcional.

Referencias

Joyanes Aguilar, L. (2020). *Fundamentos de programación: algoritmos, estructuras de datos y objetos* (5a ed.). McGraw-Hill Interamericana.

Kanagusico Hernández, E. D., Moreno García, R. C., y Hernández Sánchez, E. (2026). *Diseño de algoritmos* [Apunte digital, Licenciatura en Informática]. Universidad Nacional Autónoma de México, Facultad de Contaduría y Administración.